CMSC 27200 - Spring 2025 — Lorenzo Orecchia
due 5/9/25 via Gradescope

H7.

- The assignment is due at Gradescope on 5/9/25.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using LaTeX. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work.* Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.
- Each student can request a total of up to 5 days of extensions on all homework - either for original submission or resubmission. Up to two days can be requested for a homework. If you would like to request an extension, please fill out the Homework extension form. Extensions do not carry any late penalty. If there is a life emergency that might prevent you from submitting on time not covered by this policy, email the Head TA.
- Taking into account the different focus students have coming into this course, you can choose between a more advanced theory problem or a programming assignment. **Complete either theory problem 5 or programming problem 6.**
- This is probably the wrong place, but Lissajous curves are so pretty.

Problem 1 (Help us teach algorithms better!) *This quarter, we have been working with education researchers here at UChicago (lead by a former Head TA of this course!) to improve our course. To measure the effect of these improvements, they would like to analyze some of the work you've been submitting for this class. If you consent, we will completely anonymize your work before sending it to them.*

To earn credit for this problem, click on this link to read more about the study and indicate whether or not you consent for them to analyze your work. We will only be told ***whether*** *you have filled out the form, and not* ***how*** *you responded – your grade and experience in this course will be in no way affected by how you respond. If you have questions or concerns about the study, feel free to direct them to Konstantinos or to the study's lead (Jonathan Liu, jonliu@uchicago.edu).*

Problem 2 (Loading...) *There are N jobs, and each needs to run from time exactly s_i to t_i. The CS department decided that it wants all jobs to be run, but to do so with the least number of machines, where each machine can only run one job at a time. You will design a greedy algorithm to find the schedule of jobs that uses the minimum number of necessary machines.*

(a) *Describe the first greedy algorithm/heuristic that came to mind.* ***This does not have to be the correct algorithm!***

(b) *Describe a greedy algorithm that finds the optimal schedule of jobs that uses the minimum number of machines. Your algorithm may be stated in words or with pseudocode.*

(c) *Provide a formal proof of its correctness. You can use either "greedy stays ahead" or "exchange argument".*

(d) *Revisit your answer from part (a). If it ended up being correct, describe how you verified to yourself that it was correct. If it ended up being incorrect, describe how you determined that it was incorrect.*

Collaborators:

Solution:

PROBLEM 3 (SCHEDULE PLANNING) *In this question, you will design a dynamic programming algorithm that solves the following problem: Given the schedule of talks for the conference at each hotel, and their associated values* $\{v_{i,h}\}_{i=1}^{n}$ *for* $h = 1, 2, 3$*, as well as the costs of taking Ubers* $\{c_{h,k}\}_{h,k=1,2,3}$ *find the maximum value of any choice of talks to attend.*

Note:

- *The conference might last multiple days, but you have been allocated a room at each of the three hotels, so you can stay for free overnight in any of these three places.*
- *On the first day you can get a free Uber to whatever hotel you want to start in.*
- *You may assume that the price of Uber is independent of the direction* ($c_{h,k} = c_{k,h}$)*, though your algorithm will likely generalize to a version of this problem which doesn't satisfy this property.*

Example: *Suppose that the value of the talks are given by the following table:*

Schedule Slot:	*1*	*2*	*3*	*4*
Hotel 1	*1*	*3*	*20*	*1*
Hotel 2	*1*	*30*	*1*	*2*
Hotel 3	*15*	*3*	*4*	*3*

and let the Uber prices be given by: $c_{1,2} = 3$, $c_{1,3} = 2$ *and* $c_{2,3} = 4$*. Then the optimal schedule is given by: starting out at hotel 3, moving to hotel 2 for the second talk, moving to hotel 1 for the third talk and then staying there for the last talk. The value of this sequence of talks is* $15 + 30 + 20 + 1 - 4 - 3 = 59$*: the sum of the values of the talks attended minus the cost of the two Ubers. So your algorithm would output 59.*

(a) In general, a good subproblem to try is always a smaller version of the original problem; i.e. $DP[i]$ *is equal to the best schedule considering only days 1 to i. A natural recurrence relation would then be*

$$DP[i+1] = max_{j \in \{1,2,3\}} \left(DP[i] + v_{i+1,j} - c_{j,\text{ location at time } i} \right).$$

Describe why this recurrence relationship intuitively could be correct.

(b) Provide a counterexample to demonstrate that the recurrence relation from part (a) is in fact incorrect. (Hint: You only need two talks!)

(c) Define a 2D DP subproblem that will account for the issue brought up by your counterexample.

(d) Give a recurrence relationship for your subproblem.

(e) Argue formally that your recurrence relationship is true.

(f) Give the pseudocode of an algorithm that solves this problem based on your prior answers. Your algorithm should run in time $O(n)$ *where n is the number of talks taking place at each hotel (i.e. the number of time slots in which a talk could be scheduled). Sketch a proof that the algorithm you give satisfies this runtime bound. How much space does your algorithm use?*

Collaborators:

Solution: Your solution here.

Algorithm 1 `Optimal_Value_from_Conference`$(\{v_{i,h}\}, \{c_{h,k}\})$.

Your algorithm here.

Problem 4 (Lorenzo's Trip: Revenge of the Gas Engines) *Lorenzo is going on another trip! This time, he's back to his gasoline-powered car. He has already figured out the path he is going to take, and knows that there are gas stations $g_1, \ldots, g_n$ along the path (in that order). Lorenzo may buy a whole number (i.e. an integer) of gallons of gas at each station, and pays c_i per gallon at station g_i. Lorenzo starts at g_1 with 0 gallons and ends at g_n. His tank holds at most C gallons of gas at once, and he gets m miles per gallon. Station g_i is distance d_i away from g_1, and the distance $|d_i - d_{i-1}|$ is a multiple of m but less than Cm for every $i > 1$. The problem we are interested in is the following: given $g_1, \ldots, g_n$, $c_1, \ldots, c_n$, $d_1, \ldots, d_n$, C and m, find how much gas Lorenzo would need to buy at each station to reach g_n while minimizing the amount he pays for gas.*

(a) *Define a subproblem and give a recurrence relation that will help you solve this problem, and sketch a proof that it is correct.*

(b) *Describe an algorithm that solves this problem based on your answer to (a), and state its runtime (with sketched proof). Note that a description of any Dynamic Programming algorithm entails:*

- *The recurrence relation,*
- *The base cases,*
- *An explicit ordering in which the algorithm should evaluate the subproblems,*
- *How to extract the final answer from the completed table.*

Collaborators:

Solution: Your solution here.

You can choose to complete this theory problem **or** the programming assignment

PROBLEM 5 (CARD GAME) *A new card game is taking campus by storm! This game is played using a specialized deck, with number cards (which can represent any integer) and wild cards. Cards are placed face up in two rows of n cards each. You need to remove cards to make the two rows match while achieving the maximum score possible. You are* ***only*** *allowed to remove cards; you are otherwise not allowed to move the cards, i.e.,* ***the order of the remaining cards in each row must be preserved.*** *Scoring works as follows, working from left to right:*

- *A pair of matching number cards* ***adds*** *that number to your score. Of course, if the cards are negative, this would lower your score.*
- *Wild cards can match with any number card. In this case, you* ***multiply*** *the current score by that number, instead of adding.*
- *If you match two wild cards, you must choose whether to multiply the current score by* 25 *or* -25.

Suppose the two rows are as follows. $*$ *indicates a wild card.*

8	-3	-10	-9	-7	-5
2	-10	8	-7	-9	*

The best solution, for a total of 133 *points, is to match the* -10 *cards, the* -9 *cards, and the wild card to the* -7.

_	_	-10	-9	-7	_
_	-10	_	_	-9	*

Move	Action	Total points
-10 matches -10	add -10	-10
-9 matches -9	add -9	-19
-7 matches *	multiply -7	133

Table 1: Explanation of solution to example.

The two rows of cards are given in arrays $A[1 \ldots n]$ *and* $B[1 \ldots n]$.

In this problem, you will develop a ***dynamic programming*** *algorithm to solve this problem, and output the maximum possible score matching. Your algorithm must run in* $O(n^2)$ *time.*

First, though, consider a ***restricted version*** *of the problem, where all number cards have* ***positive*** *value. Wild cards can still appear; if you match two wildcards, you multiply the current score by* 25.

(a) *Define the subproblem that you will use to solve the* ***restricted*** *problem* ***precisely****. Define any variables you introduce.*

(b) *Give a recurrence which expresses the solution to each subproblem in terms of the solutions of smaller subproblems. Specify any base case(s).*

(c) *Write* ***pseudocode*** *for an algorithm that returns the highest scoring matching. Your algorithm must run in* $O(n^2)$ *time.*

Hint: Use backtracking.

Now consider generalizing your algorithm to the ***full version*** *of the game, in which number cards can have negative values, and matching two wild cards lets you multiply the score by* 25 *or* -25.

(d) *Give a new recurrence which expresses the solution to each subproblem in terms of the solutions to smaller subproblems. Specify any base case(s). If necessary, you can also define new subproblems.*

Collaborators:

Solution: Your solution here.

You can choose to complete this programming problem **or** the previous theory assignment

PROBLEM 6 (RIVER CROSSING) *Along the Chicago river there are many boats that want to cross from one side to the other. However, making sure they do not collide is very difficult. The harbor master has decided that in order to avoid even the possibility of collisions, he will only allow boats that have no chance to collide because their paths do not intersect. Every boat's path has two endpoints x and y on either side of the river. It is guaranteed that all the x values are unique, and all the y values are unique. You need to determine the maximum numbers of boats that can cross at the same time, as well as which boats to allow.*

A number of test cases have been provided and you can test your implementation by using the following command.

```
for i in {00..12}
do
    time python3 river_crossing.py <river_crossing_testcases/input${i}.txt |
        diff river_crossing_testcases/output${i}.txt -
done
```

The test cases and expected outputs can be found here. If you want, you may add additional test cases by adding to this folder.

In the first line of each input file there is one integer, N, the number of boats. In the next N lines there will be two integers, x, the coordinate on the left side and y, the coordinate on the right side.

The output files each contain a single integer, which is the maximum number of boats that can safely cross. Your algorithm must determine that number. Note that you do not need to do any parsing of these files yourself—that is handled for you by our tests.

Example Test Case and Explanation

The first test's input, `testcases/input00.txt` *follows:*

```
5
2 4
3 3
1 2
4 1
5 5
```

This means there are 5 *boats to consider. The first one is given as* `(2 4)`*, so it is at position* 2 *on the starting side of the river, and aims to reach position* 4 *on the ending side. It's x value is therefore* 2 *and it's y is* 4*.*

The output file, `testcases/output00.txt`*, follows:*

```
3
```

This means that 3 *boats can cross, without intersecting. Those boats are the* `(3 3)`*, the* `(1 2)`*, and the* `(5 5)` *boat. The* `(2 4)` *boat could not be included, since it would intersect with the* `(3 3)` *boat. (However, you could send the* `(2 4)` *boat in place of the* `(3 3)` *boat. This shows it is possible there are multiple optimal choices for boats to send).*

Your algorithm should therefore return 3*, since it is possible for the boats at indices* 1*,* 2*, and* 4 *to cross safely.*

Limits and Notes

The number of boats, N, will be at least 1 *and at most* 10^3*.*

The coordinates x and y will be positive, but at most 10^9*. You are guaranteed that all the x coordinates are unique, and all of the y coordinates are unique.*

The time your code has to pass any one test is 4 *seconds. The tests will timeout if your code takes longer than this time. Different machines run at different speeds, so the machine that matters here is the one that gradescope uses when it tests your code after a commit (this is the output we will read when we grade your submission).*

Submission

Upload a single python file. No other files are required.